

AOS Assignment-1 Part-A

Bratin Mondal 21CS10016

Datta Ksheeraj 21CS30037

Objective:

To configure, build, and install a customized Linux kernel from the source, focusing on specific modifications such as the removal of the IPv6 protocol and integrating the Kernel-based Virtual Machine (KVM) as an inbuilt feature, rather than as a loadable kernel module.

IPV6

The Internet Protocol (IP) is the foundational communication protocol in the TCP/IP suite, responsible for routing data packets between devices across different networks. IPv6, or Internet Protocol version 6, is the latest version designed to replace the older IPv4 protocol. One of the most significant improvements in IPv6 is its expanded address space, moving from the 32-bit address system in IPv4 to a 128-bit address system. This expansion provides a virtually limitless number of unique IP addresses, which is crucial as the number of internet-connected devices continues to grow exponentially.

IPv6 also introduces a simplified header structure, reducing the complexity of packet processing and improving routing efficiency. Unlike IPv4, which often requires additional fields and extensions, the streamlined IPv6 header helps routers process packets faster, leading to improved network performance. Another key feature of IPv6 is its mandatory support for IPsec, a suite of protocols for securing internet communications by authenticating and encrypting each IP packet. This built-in security ensures that IPv6 is inherently more secure than IPv4, which requires optional add-ons for encryption.

Additional features of IPv6 include better support for mobile devices through Mobile IPv6, which allows devices to move from one network to another while maintaining a permanent IP address. IPv6 also includes capabilities like multicast and anycast, which improve the efficiency of data transmission to multiple destinations.

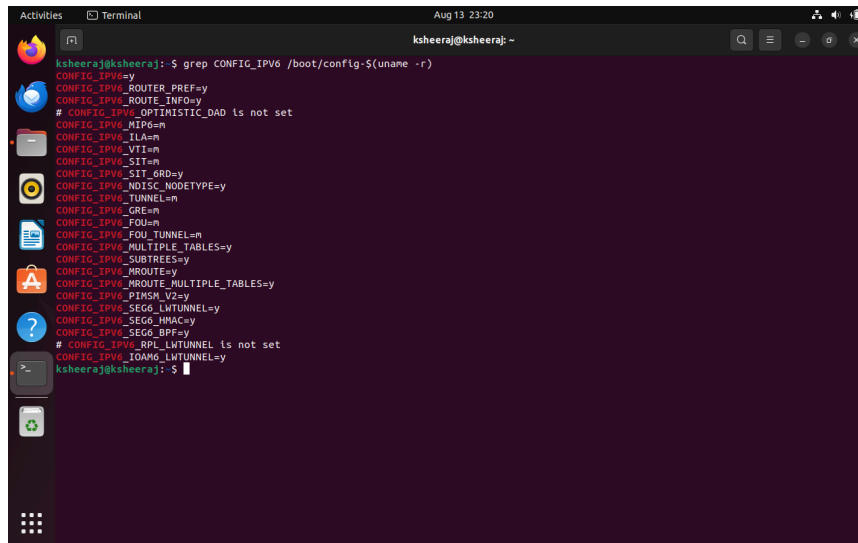
Specifically turn off the IPV6 support in the kernel configuration by navigating to the following path in the menuconfig

Networking support --->

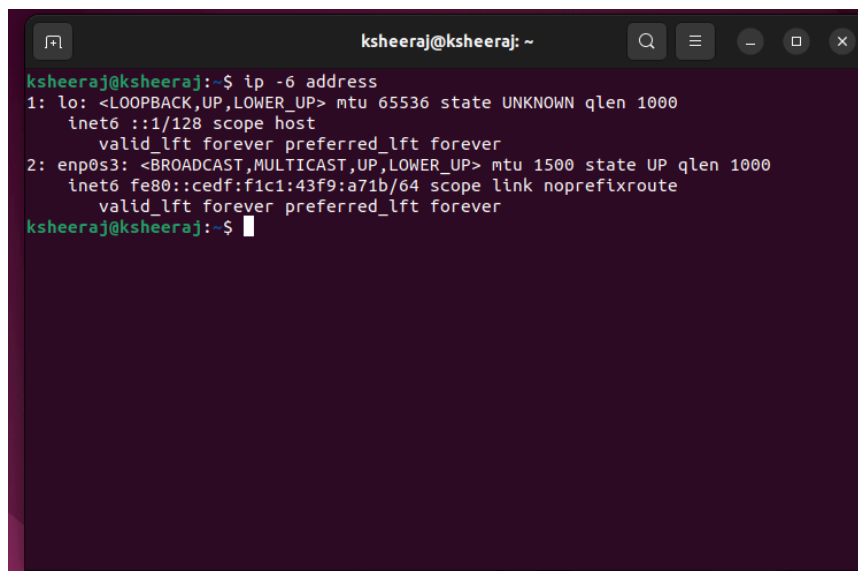
Networking options --->

<> The IPv6 protocol

Before Installation of kernel:

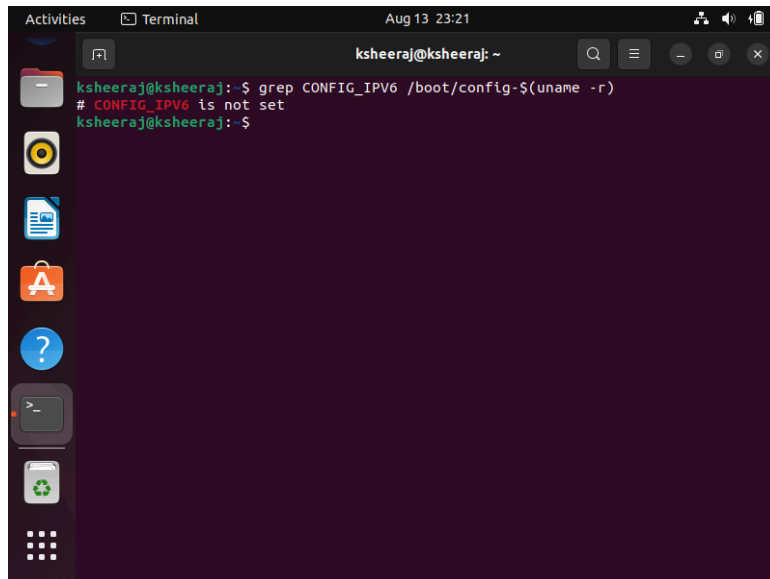


```
ksheeraj@ksheeraj:~$ grep CONFIG_IPV6 /boot/config-$(uname -r)
CONFIG_IPV6=y
CONFIG_IPV6_ROUTER_PREF=y
CONFIG_IPV6_ROUTE_INFO=y
# CONFIG_IPV6_OPTIMISTIC_DAD is not set
CONFIG_IPV6_MLPP=y
CONFIG_IPV6_ILA=m
CONFIG_IPV6_VTI=m
CONFIG_IPV6_SIT=m
CONFIG_IPV6_SIT_0RD=y
CONFIG_IPV6_NOISY_NODETYPE=y
CONFIG_IPV6_TUNNEL=m
CONFIG_IPV6_GRE=m
CONFIG_IPV6_FOU=m
CONFIG_IPV6_FOU_TUNNEL=m
CONFIG_IPV6_MULTIPLE_TABLES=y
CONFIG_IPV6_SUBTREES=y
CONFIG_IPV6_MROUTE=y
CONFIG_IPV6_MROUTE_MULTIPLE_TABLES=y
CONFIG_IPV6_PIMSM_V2=y
CONFIG_IPV6_SEG6_LWTUNNEL=y
CONFIG_IPV6_SEG6_HMAC=y
CONFIG_IPV6_SEG6_BPF=y
# CONFIG_IPV6_RPL_LWTUNNEL is not set
CONFIG_IPV6_IQAM6_LWTUNNEL=y
ksheeraj@ksheeraj:~$
```



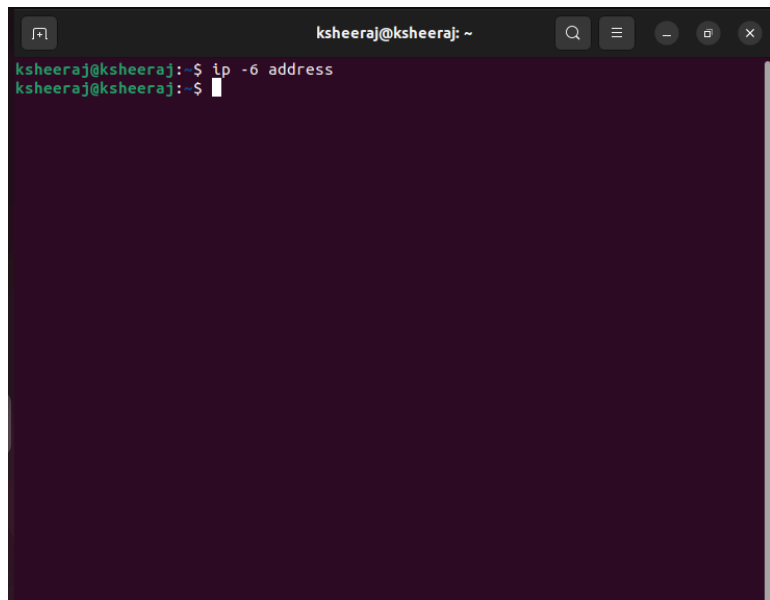
```
ksheeraj@ksheeraj:~$ ip -6 address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN qlen 1000
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 fe80::cedf:f1c1:43f9:a71b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ksheeraj@ksheeraj:~$
```

After Installation of Kernel:



A terminal window titled "Terminal" with a dark background. The window shows the following commands and output:

```
ksheeraj@ksheeraj: ~  
ksheeraj@ksheeraj:~$ grep CONFIG_IPV6 /boot/config-$(uname -r)  
# CONFIG_IPV6 is not set  
ksheeraj@ksheeraj:~$
```



A terminal window titled "Terminal" with a dark background. The window shows the following commands and output:

```
ksheeraj@ksheeraj: ~  
ksheeraj@ksheeraj:~$ ip -6 address  
ksheeraj@ksheeraj:~$
```

KVM (Kernel Virtual Machine)

Kernel-based Virtual Machine (KVM) is a vital component of Linux's virtualization capabilities, turning the Linux kernel into a type-1 (bare-metal) hypervisor. This allows multiple virtual

machines (VMs) to run on a single physical host, each with its own isolated environment, including virtualized hardware components like CPU, memory, and network interfaces.

KVM is tightly integrated with the Linux kernel, which means it benefits from all the performance, scalability, and security features that Linux offers. It supports both hardware-assisted virtualization (leveraging CPU extensions like Intel VT-x and AMD-V) and paravirtualization, where the guest operating system is aware of the virtualization environment and can interact with the hypervisor more efficiently.

One of the strengths of KVM is its ability to manage resources dynamically. It can allocate CPU, memory, and I/O resources to VMs based on their needs, ensuring optimal performance across all VMs running on the host. KVM also supports live migration, allowing VMs to be moved between hosts without downtime, which is crucial for maintaining service availability in production environments.

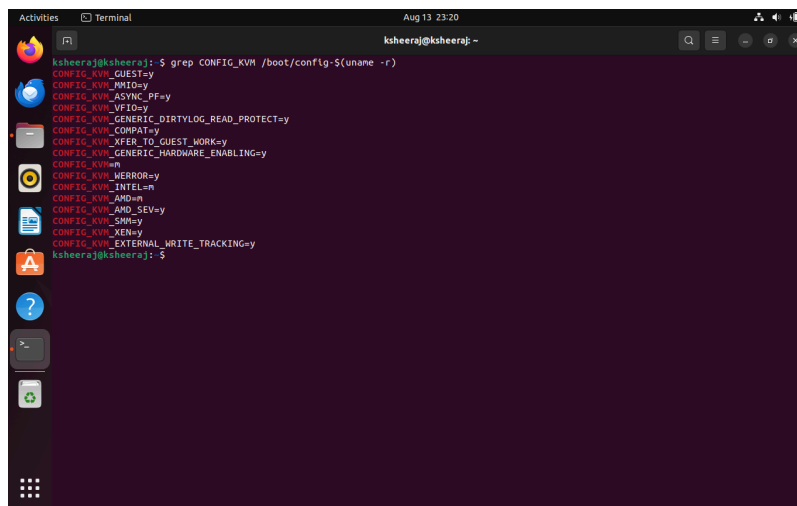
For developers and administrators, KVM provides extensive tooling and APIs, such as libvirt, for managing and automating virtualized environments. Additionally, KVM's integration with the Linux kernel ensures it receives regular updates and improvements, making it a robust and reliable solution for virtualization.

To make KVM an inbuilt feature instead of a kernel module, navigate to the following path in the menuconfig

Virtualization --->

<*> Kernel-based Virtual Machine (KVM) support

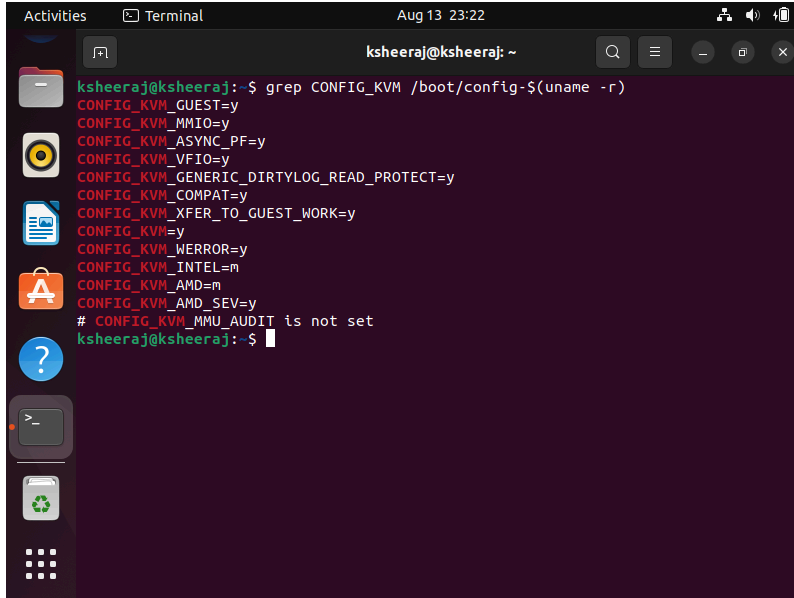
Before Installation of kernel



```
ksheeraj@ksheeraj:~$ grep CONFIG_KVM /boot/config-$(uname -r)
```

The terminal window shows the command being executed. The output lists various KVM-related configuration options, all set to 'y' (yes), indicating that KVM support is enabled in the kernel configuration. The options include CONFIG_KVM_GUEST=y, CONFIG_KVM_MMIO=y, CONFIG_KVM_ASYNC_PF=y, CONFIG_KVM_VFIO=y, CONFIG_KVM_GENERIC_DIRTYLOG_READ_PROTECT=y, CONFIG_KVM_COMPAT=y, CONFIG_KVM_XFER_TO_GUEST_WORK=y, CONFIG_KVM_GENERIC_HARDWARE_ENABLING=y, CONFIG_KVM=y, CONFIG_KVM_ERROR=y, CONFIG_KVM_INTEL=m, CONFIG_KVM_AMD=m, CONFIG_KVM_AMD_SEV=y, CONFIG_KVM_SMM=y, CONFIG_KVM_XEN=y, and CONFIG_KVM_EXTERNAL_WRITE_TRACKING=y.

After Installation of kernel



```
ksheeraj@ksheeraj:~$ grep CONFIG_KVM /boot/config-$(uname -r)
CONFIG_KVM_GUEST=y
CONFIG_KVM_MMIO=y
CONFIG_KVM_ASYNC_PF=y
CONFIG_KVM_VFIO=y
CONFIG_KVM_GENERIC_DIRTYLOG_READ_PROTECT=y
CONFIG_KVM_COMPAT=y
CONFIG_KVM_XFER_TO_GUEST_WORK=y
CONFIG_KVM=y
CONFIG_KVM_ERROR=y
CONFIG_KVM_INTEL=m
CONFIG_KVM_AMD=m
CONFIG_KVM_AMD_SEV=y
# CONFIG_KVM_MMU_AUDIT is not set
ksheeraj@ksheeraj:~$
```

Justification for some changes done:

1. System trusted keys and revocation keys are completely set to none

CONFIG_SYSTEM_TRUSTED_KEYS:

- This option specifies the file that contains keys to be added to the system keyring at boot. These keys are used to verify the signatures of kernel modules or firmware to ensure they are from trusted sources.
- Setting this option to **none** means that no keys are added to the system keyring. As a result, the kernel will not automatically trust any external modules or firmware unless explicitly configured to do so later.
- We used this to ensure that the system faces no security checks as we are customizing our kernel

CONFIG_SYSTEM_REVOCATION_KEYS:

- This option specifies the file that contains keys that have been revoked and should no longer be trusted by the kernel.
- Setting this to **none** means no revocation keys are provided, so the kernel will not revoke trust in any keys by default.
- As mentioned earlier too we used this to ensure that the system faces no security checks as we are customizing our kernel

-
2. We also changed grub timeout so as to ensure that we get to choose our own kernel while start or booting.